TECHNICAL REPORT

AlphaGlyph: A Backtesting Laboratory That Measures Its Own Credibility

Walk-forward simulation, Deflated Sharpe, and combinatorial purged cross-validation — and a 72% probability that its own best strategy is overfit.

Daniel Lichtenberger · August 2026

alphaglyph.org · Source on GitHub

A backtesting laboratory built to measure how much its own results should be believed.

Summary

Any strategy that looks good in a backtest looks good for one of two reasons: it found something real, or it was the luckiest of the many variants that were tried. The backtest itself cannot tell you which. That ambiguity is not a detail — it is the central problem of quantitative finance, and most tooling ignores it entirely.

AlphaGlyph is a backtesting engine with the second question built in. It simulates strategies on real historical prices, then subjects every result to the statistical tests that quantify selection bias: the Probabilistic and Deflated Sharpe Ratios, Combinatorially-Symmetric Cross-Validation for the Probability of Backtest Overfitting, Fama-French factor decomposition, and a forward-only prediction ledger that cannot be retroactively edited.

Pointed at its own strategy grid, the system reports a **72% probability of backtest overfitting** (SPY, run 2026-08-01) — the in-sample winner lands in the bottom half out-of-sample more often than not. That is the headline result, and it is a negative one.

- 19 REST endpoints, 302 automated tests in CI
- Indicators implemented from first principles (SMA, EMA, RSI, MACD, ADX, Bollinger)
- Live at alphaglyph.org

1. The problem: a backtest is a contaminated experiment

A backtest looks like an experiment, but it usually violates the rule that makes experiments meaningful: the hypothesis is chosen after seeing the data.

The mechanism is simple. Suppose you test 42 variants of a moving-average strategy and keep the one with the best Sharpe ratio. Even if none of the 42 has any genuine edge, the *maximum* of 42 noisy estimates is systematically above zero. You will always find a winner. Its in-sample Sharpe is not an estimate of its true performance — it is an estimate of true performance *plus a selection premium*, and nothing in the backtest separates the two.

The standard response is to report the winner's Sharpe ratio and move on. AlphaGlyph treats that as the bug, and the rest of this report describes the instrumentation built to measure it.

---

2. The demonstration: the Data-Mining Lab

Before arguing that selection bias is severe, the system demonstrates it on strategies that are *known* to have no edge.

`backend/datamining.py` generates N random long/flat timing strategies on a real price series. Each goes long on a random ~50% of days. By construction, none of them contains any information about future prices — every one is a coin flip.

The lab then backtests all N, keeps the single best Sharpe ratio, and reports two verdicts side by side:

Verdict	What it assumes	Typical result
Naive PSR	the winner was the only strategy tried	often "significant"
Deflated Sharpe	N strategies were tried; corrects for it	"not significant"

The gap between those two numbers is the entire lesson. The winning strategy is 100% luck — this is guaranteed by how it was generated — yet naive statistics frequently call it significant. The expected maximum Sharpe of N noise strategies is drawn on the histogram, and the winner sits exactly where chance predicts.

This is the argument the rest of the system is built to act on: **a Sharpe ratio is meaningless without knowing how many were tried to find it.**

---

3. Probabilistic and Deflated Sharpe Ratios

Implemented in `backend/stats.py`, following López de Prado (2014).

3.1 PSR

The naive Sharpe ratio assumes returns are normal and the sample is large. Financial returns are neither — they are skewed and fat-tailed, which makes the Sharpe estimate noisier than it appears. PSR returns the probability that the *true* Sharpe exceeds a benchmark, correcting for both:

```
PSR(SR*) = Φ[ (SR̂ − SR*) · √(T−1) / √(1 − γ₃·SR̂ + (γ₄−1)/4 · SR̂²) ]
```

where γ_3 is skewness and γ_4 raw kurtosis. Since SciPy reports *excess* kurtosis, the implementation uses `(excess + 2)/4`, which is the same quantity. All Sharpe values are held in daily units internally and converted only at the boundary — a detail that is easy to get wrong and silently changes the answer.

3.2 DSR

The Deflated Sharpe Ratio is PSR with the benchmark raised to the Sharpe you would expect from the luckiest of N independent trials:

```
SR*_annual = E[max Z_k] · √(252 / T)
```

`E[max Z_k]` uses the standard Euler-Mascheroni approximation to the expected maximum of N unit normals.

The `√(252/T)` scaling matters and is frequently omitted in third-party implementations. The expected maximum is expressed in units of the *sampling distribution* of the Sharpe estimate, whose width depends on how many observations you have. Without rescaling for T, the deflation is applied at the wrong magnitude and the test is miscalibrated. With `n_strategies = 1`, DSR reduces exactly to PSR against a zero benchmark, which is the correct degenerate case and is asserted in the tests.

---

4. CSCV and the Probability of Backtest Overfitting

`backend/cpcv.py`. Method from Bailey, Borwein, López de Prado & Zhu (2017), via Combinatorially-Symmetric Cross-Validation (CSCV). An earlier version of this report called it CPCV, which is a different method: López de Prado's Combinatorial *Purged* CV builds many backtest paths, while CSCV measures how often the in-sample winner fails out of sample. The module and the `/api/research/cpcv` route keep the old name so existing links work.

4.1 The strategy grid

The grid is deliberately made of strategies people actually tune, not strawmen:

- **Trend-following** — long when fast SMA > slow SMA, else flat. Fast ∈ {5, 10, 15, 20, 30, 40}, slow ∈ {50, 80, 100, 150, 200}, keeping only fast < slow → 30 variants.
- **Mean-reversion** — stateful RSI entry/exit: enter below a buy threshold, exit above 55. Period ∈ {7, 14, 21}, threshold ∈ {25, 30, 35, 40} → 12 variants.

42 variants total. Positions are shifted one day, so a decision made on the close of day t is held over $t \to t{+}1$. There is no look-ahead. The first 205 rows are dropped to clear the longest indicator's warm-up.

4.2 The procedure

1. Build a T×N matrix of per-period returns, one column per variant.
2. Split the T rows into S contiguous groups (S even, clamped to 6–12; C(12,6) = 924 combinations is the computational ceiling).
3. For every way of choosing S/2 groups as in-sample:
    - pick the variant with the best in-sample Sharpe, `n*`
    - find `n*`'s **rank** among all variants out-of-sample
    - ω = rank / (N+1); logit $\lambda = \ln(\omega / (1-\omega))$
4. **PBO** = the fraction of splits where $\lambda \le 0$ — i.e. the in-sample champion lands in the bottom half out-of-sample.

A PBO near 0.5 means selection told you nothing at all.

4.3 Purging and embargo

Financial returns are serially correlated, so an in-sample block sitting immediately adjacent to an out-of-sample block can leak its edge straight across the seam. The implementation trims `embargo` rows from each end of every in-sample block — and critically, **only** from the in-sample side. Trimming both would discard out-of-sample data that is legitimately available and bias the result optimistically.

4.4 Degenerate columns

A variant that stays flat across an entire block has zero variance and therefore an undefined Sharpe. These are assigned `-inf` so they can never be selected as the in-sample champion, and are filtered out before the degradation regression, medians and scatter. Silently letting a no-trade outcome count as a performance figure would distort every downstream statistic — this is the kind of edge case that turns a correct method into a wrong number.

4.5 What it also reports

- **Performance degradation** — OLS of OOS Sharpe on IS Sharpe. A slope well below 1 means the in-sample edge mostly evaporates.
- **Probability of OOS loss** — how often the selected strategy actually loses money.
- Logit histogram, IS-vs-OOS scatter, and a split diagram for the interface.

4.6 Calibrated by simulation

Checking a formula on one worked example shows the code matches the paper. It does not show the statistic does its job. `backend/tests/test_stats_calibration.py` checks the property each statistic exists for, on simulated data where the right answer is known (seeded, two years of daily returns):

Check	Should be	Measured
PSR at 95% calls pure noise significant (normal returns, 2,000 draws)	about 5%	**5.05%**
Same, fat-tailed returns (Student-t, 4 df)	about 5%	**5.1%**
Best of 20 noise strategies: PSR calls the winner significant	often (selection bias)	**63%**
Best of 20 noise strategies: DSR calls the winner significant	under 5%	**0.5%**
PBO when all 30 strategies are noise	about 0.5	**0.49**
PBO when one of 30 has a real edge (Sharpe ≈ 2.4)	near 0	**0.07**

The middle rows are the reason the Deflated Sharpe Ratio exists: picking the best of 20 coin flips and testing it as if it were the only one fools the plain test most of the time. DSR is conservative here, because it tests against the *expected* best of N. To make the simulation possible, PBO's core was split from the price-driven grid into `pbo_from_matrix`, which accepts any matrix of strategy returns.

---

5. Factor decomposition

A strategy can post real returns and still have no alpha, if it is merely capturing well-known risk premia that a passive factor ETF sells for a few basis points. `fama_french_decomposition` regresses daily excess returns on the three Fama-French factors:

```
R_p − R_f = α + β_mkt(R_m − R_f) + β_smb·SMB + β_hml·HML + ε
```

and reports Jensen's alpha annualised, factor loadings, R^2, and per-coefficient t-statistics. Factors are the real daily series from Ken French's data library, not a proxy.

An engineering note worth recording: this test kept intermittently reporting "n/a" in production. Two causes, both environmental rather than statistical — Ken French had moved the file from `/data_library/` to `/ftp/`, and Dartmouth blocks the default `python-requests` user agent from some cloud IPs. The fix was a browser-like UA, both URLs tried in order, retries with backoff, and a disk cache of the last good copy as an offline fallback. A statistical test is only as reliable as its data pipeline, and this one was failing for reasons that had nothing to do with statistics.

---

6. Calibration and the forward ledger

Two components address a claim the backtest cannot make: whether the forecaster's *probabilities* are honest.

`calibration.py` — the model emits statements like "~58% chance of an up move." A forecaster is well-calibrated when, across every day it said 60%, the market actually rose about 60% of the time. Predictions are binned into deciles to produce a reliability diagram and scored with Brier score and log loss, each against a "always predict the base rate" baseline so the number means something.

`ledger.py` — the harder test. Every directional call is written to an append-only table *at the moment it is made*, with price, p_up, direction and horizon. Once the horizon elapses, the row is graded against what actually happened. **Nothing is ever edited or deleted.** The resulting hit rate and Brier score are a genuine forward record that cannot be retro-fitted.

Calibration measures the model on historical out-of-sample data already in hand; the ledger measures it on the future, one real day at a time. Grading is lazy — a plain GET matures any elapsed rows, so the record stays current without a cron job.

The live ledger turned out not to accrue. After three months it held three graded calls. The backend runs on Render's free tier with no database configured, so the table lived in a SQLite file on an ephemeral disk that every redeploy wiped, and it only recorded predictions a visitor happened to ask for. A forward record that resets is not a forward record.

So the forward record now lives in git (`tools/forward_ledger.py`, run by `.github/workflows/forward-ledger.yml`). Every trading day near the close, the live Predictor is asked about a fixed basket of ten tickers (SPY, QQQ and eight large caps), and each call is appended to `frontend/data/forward_ledger.csv` and committed. Calls whose horizon has passed are graded against the first close on or after the due date and filled in once. The commit history is the audit trail: each call sits in a commit dated before its outcome existed. The first scheduled runs started about three hours late, and a logging window that closed at 16:15 ET would have recorded nothing. Calls are now accepted from 15:15 ET until midnight New York time, priced at that day's close, which is still days before the outcome. A third, later schedule is a backup. Both records grade with the same rules (`track_record.py`), and `/health` now reports which database the live ledger is using.

---

7. Results

> **Reproduce before citing.** CSCV runs live against a rolling window of fetched price data, so PBO is a function of the window it was run on. Every figure below carries the date it was produced.

CSCV on SPY — 42 variants, 10 groups, 252 splits, 1,050 rows after warm-up, embargo 5. Run 2026-08-01:

Reproducible from committed data: the window this run used (SPY closes, 2021-08-02 → 2026-07-31) is committed at `docs/data/SPY_close_2021-08-02_2026-07-31.csv`, and `python tools/pbo_snapshot.py` recomputes every figure in this table from it, through the same `cpcv_from_prices` the live endpoint calls. Re-downloading the same dates would not be the same data: Yahoo revises adjusted closes as later dividends are paid.

Metric	Value	Reading
Probability of backtest overfitting	**72%**	selection is worse than a coin flip
Median in-sample Sharpe	**1.26**	looks like a good strategy
Median out-of-sample Sharpe	**0.65**	~52% of the edge survives
IS→OOS degradation slope	**−0.59**	see below
Probability of OOS loss	**5%**	it rarely *loses* money
ML forecaster test AUC	**≈0.51**	a coin flip on next-week direction

The window-dependence is itself a result

An earlier run of the identical procedure reported **PBO 63%** with **~38%** Sharpe retention. This run reports **72%** and **~52%**. Same code, same grid, same ticker — a different rolling window.

That instability is not a defect in the measurement; it is the measurement working. A single PBO number is a statement about one window, and anyone quoting it as a fixed property of a strategy has made the same category error the metric exists to catch. This report therefore dates every figure.

The degradation slope is negative

The regression of out-of-sample Sharpe on in-sample Sharpe has a slope of **−0.59**. A slope near 1 would mean in-sample performance predicts out-of-sample performance; a slope near 0 would mean it predicts nothing.

A *negative* slope means the relationship is inverted — across these splits, the variants that looked best in-sample tended to do **worse** out-of-sample. Selecting on in-sample Sharpe was actively worse than selecting at random.

What the 5% OOS-loss rate means

Worth stating precisely, because it is easy to over-claim. The selected strategy loses money out-of-sample only about 5% of the time. The failure is not that these strategies are ruinous — it is that **choosing among them on in-sample performance does not identify the good ones.** The money is mostly fine; the selection procedure is what's worthless.

The one clean positive, with its conditions: a dip-buying strategy on AAPL/MSFT from 2021-01-01 to 2024-01-01 returned **+21.6%** in ranging markets against SPY's **+5.6%** over the same days — an excess of +16.1 percentage points. Both conditions are load-bearing. The regime one is the obvious one; the window is the other: extending the run to 2025-01-01 shrinks the ranging-market excess to **+3.6** points, because 2024's ranging days rewarded simply holding the index. It is not a claim that the strategy works in general. (Rerun on 2026-10-27; the July figures, +21.8% vs +5.6%, differed only by data revisions.)

The AUC of 0.51 is worth stating plainly. The machine-learning forecaster, which took the most work to build, cannot predict next-week direction. It is reported at 0.51 rather than quietly dropped, because a system whose entire premise is honest measurement does not get to hide its own null results.

A modality ablation: do macro and news inputs help?

The forecaster is a transformer over 60-day windows with three input blocks: price features, macro series from FRED, and GDELT news tone. The shipped checkpoint had been trained with the macro and news blocks zero-filled, so the multi-modal design had never been tested. The ablation (`ml/ablation.py`, output in `ml/ablation.json`) retrains the same architecture, with the same splits and early stopping, on three input sets: price only, price + macro, and all three. Each is trained from three seeds, and the seeds' probabilities are averaged. An L2 logistic regression on the last day's 13 features is the baseline a deep model has to beat.

Training data runs to 2020-12-23 and validation to 2023-11-29, with a 10-day purge at each boundary. Every figure below is on the test split after that: 85 tickers, 5-day direction, 60,945 windows, 54.2% of them up. Nothing was tuned on it. Uncertainty comes from a moving-block bootstrap over test dates (20-trading-day blocks, 1,000 resamples), not over rows. The 85 tickers on one day share a market and the 5-day labels overlap, so resampling rows would make every interval far too narrow. Arms are compared on the same resampled dates. News tone is lagged one day, because GDELT buckets by UTC day and a same-day value would leak. It was found for 57 of the 85 tickers, which is 67% of test windows.

Inputs	Test AUC (3-seed ensemble)	95% CI	Resamples with AUC ≤ 0.5	Validation AUC
Price only	0.509	0.492 – 0.527	18%	0.525
Price + macro	0.502	0.477 – 0.527	44%	0.529
Price + macro + news	0.508	0.487 – 0.532	22%	0.525
Logistic regression (last day's features)	0.513	0.493 – 0.537	9%	0.528

Comparison (same resampled dates)	AUC difference	95% CI
Adding macro to price	−0.008	−0.027 – +0.015
Adding news to price + macro	+0.007	−0.007 – +0.020
All three vs price only	−0.001	−0.015 – +0.017
Transformer (all three) vs logistic regression	−0.005	−0.024 – +0.013

Three findings, all null:

1. **Neither extra input helps.** Every paired difference is under 0.01 AUC, and every interval spans zero.
2. **The transformer does not beat a linear model.** The logistic regression has the highest test AUC (0.513), though not by a margin the intervals can tell apart.
3. **Validation chose the worst arm.** Price + macro had the highest validation AUC and the lowest test AUC. With differences this small, picking on validation is close to picking at random, which is the selection problem of §4 in miniature.

The rule for this experiment was set before it ran: report the result whatever it is, and replace the shipped model only if validation picked a different input set *and* its test interval was clear of the current model's. Neither held, so the shipped model, the price-only network, is unchanged.

---

8. Limitations

- **PBO is window-dependent.** A single PBO figure describes one data window and one grid. It is not a universal constant.
- **The grid is only two strategy families.** Trend and mean-reversion are representative of what retail practitioners tune, but a wider grid could produce a different number.
- **CSCV assumes reasonably comparable variants.** With a grid mixing wildly different exposure profiles, rank-based comparison is less meaningful.
- **Transaction costs are modelled, not measured.** Real fills, slippage and market impact are not simulated.
- **Option prices in the options backtester are modelled.** Black-Scholes with a fitted volatility level and skew and the real VIX term structure, no dividends and no early assignment. Checked against six Cboe strategy indices out of sample, it is within about a point a year of each but understates the worst months for option sellers (§9.1).
- **The forward record is young.** It accrues ten calls a trading day from November 2026. Until it holds a few hundred graded calls, its hit rate and Brier skill are noise, and a single market regime dominates it.

---

9. Options strategy backtests

The options backtester extends the same question, whether an edge is real, to option strategies. It holds one position at a time on one underlying: it opens the legs, marks them to market at every close, closes on the first of expiry, a days-before-expiry rule, a profit target or a stop loss, and opens the next position the same day. Units are sized so that each position's worst case at expiry (or, for uncovered short calls, the loss on a 30% rally) uses at most a set share of the account. Idle cash earns the 3-month T-bill.

Prices are modelled, and this is the central limitation. No free source has historical option quotes, so each leg is priced with Black-Scholes on that day's close. Three choices set the prices, and §9.1 checks them against real option returns:

1. *Level.* For SPY, at-the-money implied volatility is 0.845 × the VIX. The VIX is a variance-swap rate that includes the skew, so 30-day at-the-money IV sits below it. Other tickers use their own 21-day realized volatility times the market's implied-to-realized ratio that day (clipped to 0.8-2.0), so their IV rises in fearful markets, as real IV does.
2. *Skew.* IV changes with standardized moneyness, by 0.175 per standard deviation on the put side and 0.40 on the call side, floored at 0.75 × at-the-money: a 30-day put 5% out of the money trades near 1.2 × at-the-money IV and a 30-delta call near 0.8 ×. An early version used the raw VIX with a mild symmetric skew and showed an SPY iron condor at a 26% CAGR over 2021-2025; the next used 0.40 on puts and 0.15 on calls. Both were wrong in ways §9.1 measured.
3. *Term structure.* IV at other maturities comes from the real VIX curve that day: the VIX9D, VIX, VIX3M and VIX6M give 9, 30, 93 and 183 days, interpolated linearly in total variance, and the same ratios are applied to every ticker. Before 2011, when the VIX9D starts, the short end falls back to a modelled slope, (T / 30 days)^β with β falling from about +0.10 in calm markets to below zero in panics. That slope was the first version of the term structure. Against the real curve it was right on average at 93 days (a mean ratio of 1.113 against 1.111) but too steep at 9 days (0.888 against 0.946), and it tracked the day-to-day shape loosely (correlations of 0.46 to 0.77).

Calendars depend most on shapes no index checks (the term structure between two maturities, at moving strikes), so they are the least reliable results: the SPY calendar loses 16.7% a year on the window of §9.2. Every result in the UI carries a "model-priced" banner and the volatility source.

9.1 Checked against real option returns

The model can be tested after all, because Cboe publishes the returns of real option strategies as indices: the PutWrite index on Yahoo (daily closes from 1996) and five more as CSV files on Cboe's site. Each is replicated in the backtester on SPY, by its published method, simplified: same legs, one-month options held to expiry (the collar's three-month put is rolled monthly), one unit per 100 shares' worth of SPY as the indices are sized, and no costs, since the indices have none.

Fitted on 2007-2015 only. The model's free settings were chosen on that window, each on a grid, so that a replica's annual return matched its index: the volatility level against the PutWrite index (5.86% against 5.91%, giving 0.845), then the put-side skew against the put-protection index PPUT, whose puts are 5% out of the money, and the call-side skew against the 30-delta buy-write index BXMD. Everything below is 2016-01-04 to 2026-11-06, which the fit never saw; BXM, CLL and CNDR were not used at all (`tools/options_validation.py`, output in `frontend/data/options_validation.json`):

Index	Strategy	Model	Real	Gap	Gap before the skew fit	Monthly correlation
PUT	sell a one-month at-the-money put	8.7%	8.2%	+0.6	+0.7	0.87
BXM	own the index, sell a one-month at-the-money call	8.7%	7.6%	+1.1	+1.2	0.87
BXMD	own the index, sell a one-month 30-delta call	10.1%	10.3%	−0.3	+2.9	0.93
PPUT	own the index, buy a one-month put 5% out of the money	12.2%	11.7%	+0.5	−3.4	0.95
CLL	own the index, buy a three-month 5% put, sell a one-month 10% call	9.9%	10.8%	−0.8	−0.9	0.87
CNDR	sell 20-delta puts and calls, buy 5-delta wings	0.1%	0.8%	−0.7	+4.7	0.45

The gaps are the model's annual return minus the real index's, in points. Before the skew fit, with 0.40 on puts and 0.15 on calls, the model priced out-of-the-money puts too richly and out-of-the-money calls too richly as well: protective puts cost 3.4 points a year too much, 30-delta call writing earned 2.9 too much, and the iron condor, which neither fit used, earned 4.7 points a year too much. After the fit every index is within about a point a year, the condor included, which is the out-of-sample evidence that the new skew is closer to real chains.

What it does not fix: **the worst months**. The PutWrite replica's worst month was −8.4% against the index's −13.4%, and BXM's −8.4% against −14.9%. Real option prices jump further in a crash than a smooth volatility model allows, so the model **understates tail losses for option sellers**, and its results for them should be read as an upper bound. The condor's monthly correlation of 0.45 is also low: a four-legged position near its strikes is where small pricing differences matter most.

The replicas and the indices also differ by design. The indices trade SPX options at Cboe's special opening quotation on the third Friday; the replicas trade SPY options at the close every four to five weeks. The fitted settings absorb the average effect of those differences, so they are effective values, not measurements of implied volatility. Cboe's files are used for these figures only; the repository commits the derived results, not the files.

9.2 Results on SPY

On SPY over 2021-11-08 → 2026-11-06 (30 days to expiry, default sizing and costs):

Strategy (legs)	Return	CAGR	Sharpe	Max drawdown
Buy and hold SPY	+67.1%	+10.9%	0.45	−24.5%
Cash-secured put (1)	+38.1%	+6.7%	0.40	−9.9%
Covered call (2)	+29.5%	+5.3%	0.16	−19.0%
Protective collar (3)	+28.2%	+5.1%	0.15	−20.2%
Long call (1)	+22.1%	+4.1%	0.20	−49.4%
Iron condor (4)	−5.0%	−1.0%	−0.18	−34.1%
Long straddle (2)	−29.8%	−6.9%	−0.28	−47.6%

None beat buy-and-hold on return in a window where SPY rose 67%. The cash-secured put kept its drawdown to two-fifths of the market's while earning six-tenths of its return; the iron condor lost money; the straddle paid theta every month. These figures are lower for option sellers than an earlier version of this report showed (a 9.5% CAGR for the cash-secured put), because the skew fit of §9.1 made out-of-the-money options cheaper to sell. The tests check the parts that do not depend on the volatility model: Black-Scholes against a textbook value and put-call parity, expiries on Fridays, delta strikes, offsets that always form a spread, the capital rule for each structure, no look-ahead (changing later prices leaves the first trade unchanged), costs that only ever lower the result, and an exact accounting identity: final equity equals starting capital plus the sum of every trade's P&L.

9.3 Where the P&L comes from

A return figure doesn't say what earned it. Each day a position is open, the backtester splits its change in value with the previous close's Greeks: delta × the price move, ½ gamma × the move squared, theta × the days elapsed, and, for each leg separately, vega × the change in that leg's own implied volatility (legs move by different amounts because of the skew and the term structure). What the Taylor expansion misses is reported as "other": higher-order terms, rate moves, and above all the last days before expiry, when the Greeks change fastest. By construction the parts add to each trade's P&L, and with commissions, spreads and interest on idle cash they add to the account's (both are tested).

On the same SPY window:

Strategy	Direction (delta)	Size of moves (gamma)	Time decay (theta)	Volatility (vega)	Other	Costs	Interest	Total
Cash-secured put	+$21.6k	−$48.5k	+$44.8k	−$1.8k	−$0.6k	−$0.6k	+$23.2k	**+$38.1k**
Covered call	+$31.3k	−$43.1k	+$41.0k	−$7.4k	+$6.8k	−$3.6k	+$4.5k	**+$29.5k**
Iron condor	−$9.1k	−$171.1k	+$138.2k	−$24.0k	+$48.1k	−$4.5k	+$17.3k	**−$5.0k**
Long straddle	−$22.6k	+$284.6k	−$290.9k	+$12.5k	−$25.4k	−$6.1k	+$18.0k	**−$29.8k**

Three readings the headline returns hide. The cash-secured put's +$38k includes $23k of T-bill interest on its collateral, 61% of the total: most of what put-selling "earned" here is the cash yield, which a buy-and-hold comparison doesn't separate. The iron condor collected $138k of time decay and gave back $171k on large moves. And the straddle shows the long-volatility bargain: big moves paid $285k, and time decay took $291k.

9.4 Is an options edge real?

The options lab is a new way to fool oneself, and the tools of §3 and §4 apply to it directly. They run on the options backtester's daily excess returns.

The leaderboard is 19 tries. "Compare all strategies" runs every preset and ranks them, which is a selection from 19. On the SPY window, the luckiest of 19 strategies with no edge would show a Sharpe near 0.84 by chance. The best row, a bull put spread with a Sharpe of 0.41 and a PSR of 82% on its own, has a Deflated Sharpe of 17% once judged as the best of 19. No row comes near 95%, and the leaderboard shows the deflated figure next to each Sharpe.

Settings are tries too. Options traders tune days to expiry and exit rules. The robustness test reruns one strategy at 7, 14, 30, 45 and 60 days, each held to expiry, closed at 50% profit, closed halfway to expiry, or stopped at a 100% loss: 20 backtests. It deflates the best setting's Sharpe for 20 tries and runs the CSCV procedure of §4 on the 20 return streams (10 groups, 252 splits). On SPY, 2021-11 to 2026-11:

Strategy	Best of 20 settings	Its Sharpe	Deflated Sharpe	PBO	Best setting loses out of sample
Cash-secured put	60 days, close halfway	0.77	43%	64%	19% of splits
Covered call	45 days, hold to expiry	0.43	18%	4%	16%
Iron condor	60 days, take 50% profit	0.19	7%	23%	76%
Long straddle	7 days, hold to expiry	0.43	17%	18%	50%

No best setting has a Deflated Sharpe near 95%. The two tests answer different questions. The covered call's settings rank consistently (a PBO of 4%), but its best Sharpe is no more than 20 tries of luck would produce. The cash-secured put's best setting has the highest Sharpe and the worst PBO, 64%: the setting that looks best in one half of the history usually doesn't in the other. These PBO figures move with the window as §7 found for stocks: ending the same test two days later, on 2026-11-08, puts the iron condor's PBO at 25% rather than 23%. A pass on either test would check the selection, not the prices, so the page says a pass is a reason to test with real quotes, not proof of an edge.

Two engineering notes. A sweep is 20 full backtests in one web request, so the delta-strike search, which priced all 201 candidate strikes for every new position, now bisects (|delta| is monotone in the strike) and then compares the neighbouring strikes; results are identical to the full search (tested on 300 random cases), and runs are about 2.5 times faster. Even so, a five-year sweep takes about a minute on the free hosting tier, so sweeps are limited to five years of history to stay inside the server's request time.

9.5 Entry rules, and how much of a result is the model

Entry rules. A position can be made to open only when the VIX is in a range, when the underlying's IV rank (today's implied volatility against its past year) is in a range, or when the underlying is above or below its 200-day average. The rules gate entries only; a position that is open when a rule stops passing runs to its own exit. On the SPY window, no rule improved the plain cash-secured put:

Cash-secured put, only when…	CAGR	Sharpe	Max drawdown	Days allowed
(no rule)	+6.7%	0.40	−9.9%	100%
the VIX is 20 or higher	+6.6%	0.39	−8.4%	33%
the VIX is below 20	+4.2%	0.05	−11.9%	67%
IV rank is 50 or higher	+4.3%	0.06	−13.1%	49%
SPY is above its 200-day average	+5.1%	0.17	−11.4%	78%
SPY is below its 200-day average	+5.3%	0.24	−7.3%	22%

Selling puts only when the VIX was 20 or higher kept almost all of the return with a smaller drawdown, in a third of the days, the one rule that came close. Each rule is also one more try in the sense of §9.4.

Model sensitivity. Every backtest is rerun three more times with the volatility model changed: implied volatility 10% lower throughout, 10% higher, and without the skew.

Strategy	As run	IV 10% lower	IV 10% higher	No skew
Cash-secured put	+6.7%	+4.7%	+9.4%	+5.4%
Covered call	+5.3%	+4.9%	+9.1%	+11.1%
Iron condor	−1.0%	−5.5%	+4.7%	+13.0%
Long call	+4.1%	+15.8%	−6.1%	+2.0%
Long straddle	−6.9%	+2.4%	−15.7%	−8.0%

A 10% error in the volatility level is worth more than most of the differences between strategies: it moves the iron condor between −5.5% and +4.7% a year and the long call between −6.1% and +15.8%. Short-premium returns are, roughly, the gap between implied and realized volatility, and a 10% shift in implied volatility is a large share of that gap. This is why §9.1 matters: the PutWrite check pins the level well inside 10%, since a 0.6-point annual gap on the PutWrite replica corresponds to an error of about 2 to 3% in volatility. The skew line shows why the skew fit mattered: without a skew the covered call and the condor look far better than with it. Each result shows the range next to its headline, flagged when it spans five points or more.

---

10. Reproducing

```
git clone https://github.com/Danny-397/Alphaglyph
cd Alphaglyph/backend
pip install -r requirements.txt
pytest                      # 302 tests
python app.py               # http://localhost:5000
```

Architecture: the browser holds all UI state and the backend is a stateless compute layer; the ML model is trained offline and shipped as a portable ONNX artifact. Data resolves through a three-stage cascade — Tiingo, then yfinance, then Stooq — so a single upstream outage does not take the system down.

---

11. References

1. Bailey, D. H., Borwein, J. M., López de Prado, M., & Zhu, Q. J. *The Probability of Backtest Overfitting.* **Journal of Computational Finance, 20(4), 39–70 (2017).** Circulated as an SSRN working paper from 2014; the 2014 date commonly cited refers to that preprint, not the journal publication.
2. Bailey, D. H., & López de Prado, M. (2014). *The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting and Non-Normality.* **Journal of Portfolio Management, 40(5), 94–107.**
3. Fama, E. F., & French, K. R. (1993). *Common Risk Factors in the Returns on Stocks and Bonds.* **Journal of Financial Economics, 33(1), 3–56.**

Citations verified against publication records on 2026-08-01.